AgentMesh

FAB Banking AI Agent Platform

Technical Documentation & Development Record

Version 2.0 · August 2026 · Confidential — Internal Use Only

3	4	7	18	5	3	10+
Microservices	AI Agents	RBAC Roles	Data Tools	Banking Docs	Eval Layers	Lessons

Built with:

Python 3.13 · Microsoft MAF · Groq LLM API · React 18 + TypeScript · Qdrant Vector DB · ChromaDB · MySQL · OpenTelemetry · FastMCP · Starlette

Contents

1. Executive Summary
2. System Architecture
3. Processing Pipeline — Six Stages
4. Security & Compliance Layer
5. Agent Mesh Topology & Orchestration
6. Data & Knowledge Services
7. Semantic Response Cache
8. Conversation Memory & Multi-Turn
9. AI Reasoning Explainability
10. Human-in-the-Loop (HITL)
11. Observability & Audit Trail
12. Evaluation Framework
13. Development Lessons Learned
14. Project Structure

1. Executive Summary

AgentMesh is a distributed AI agent orchestration platform built for First Abu Dhabi Bank (FAB) to deliver intelligent, compliant pricing assistance for corporate banking products. The system combines four specialised AI agents, a multi-layer security pipeline, structured data intelligence from a MySQL semantic layer, and regulatory document retrieval from a Qdrant vector database — all wired together with full audit logging, OpenTelemetry distributed tracing, and a React chat interface that streams responses in real time.

What It Solves

Corporate pricing decisions at FAB require synthesising data from multiple systems — customer 360 profiles, historical deal margins, regulatory concentration limits, CBUAE circulars, and internal policy documents — while enforcing strict role-based access controls and compliance rules. Analysts previously spent hours manually cross-referencing these sources. AgentMesh automates this synthesis in seconds while maintaining a full, immutable audit trail for regulatory review.

Platform Capabilities at a Glance

Capability	Detail
Microservices	Agent Mesh Orchestrator · DataLayer-as-Service · RAG-as-Service
AI Agents	ComplianceAgent · DataAgent · RAGAgent · PriceAssistAgent (primary orchestrator)
User Roles	7 roles: customer · relationship_manager · branch_operations_officer · credit_officer · compliance_officer · operations_manager · platform_administrator
Data Intelligence	18 MCP tools over MySQL fab_semantic views — customer 360, pricing, margins, RWA, non-compliant deals, win/loss, competitor analysis
Knowledge Retrieval	5 FAB banking PDFs in Qdrant with hybrid dense+sparse retrieval, cross-encoder reranking, RAGAS evaluation
Security	4 independent layers: regex guardrails → RBAC rules → LLM compliance gate → PII output redaction
Semantic Cache	ChromaDB vector cache with entity gating, zone-based routing, LLM gray-zone judge, paraphrase augmentation — serves hits in ~50ms
Conversation Memory	JSONL per-session history + async rolling LLM summarisation (≤200 words)
Human-in-the-Loop	Role-triggered (credit_officer) and tool-triggered approval gates with checkpoint persistence
Observability	OpenTelemetry distributed tracing, W3C baggage propagation, JSONL audit trail, per-request log files
Evaluation	3-layer framework: 20 golden workflow cases + 15 custom evaluators + 36 FinBEN/FLARE tasks + red-team suite

Capability	Detail
Frontend	React 18 / TypeScript chat with SSE streaming, AI Reasoning tab, pipeline trail, 7 dashboard pages

2. System Architecture

The platform is composed of three independently deployable Python microservices plus a central API orchestrator and a React frontend. All inter-service communication uses typed HTTP — the A2A (Agent-to-Agent) protocol for agent hops and MCP (Model Context Protocol) for tool invocation. Each agent runs as an isolated OS process on its own port, preventing any shared memory or state leakage between agents.

The critical architectural distinction is the **two-tier agent hierarchy**: the API Server calls ComplianceAgent (safety gate) and PriceAssistAgent (primary orchestrator) directly. PriceAssistAgent then sub-orchestrates DataAgent and RAGAgent internally via A2A — the API Server never calls DataAgent or RAGAgent directly. This layering gives PriceAssistAgent full control over intent classification, tool selection, and answer synthesis.

Architecture Diagram

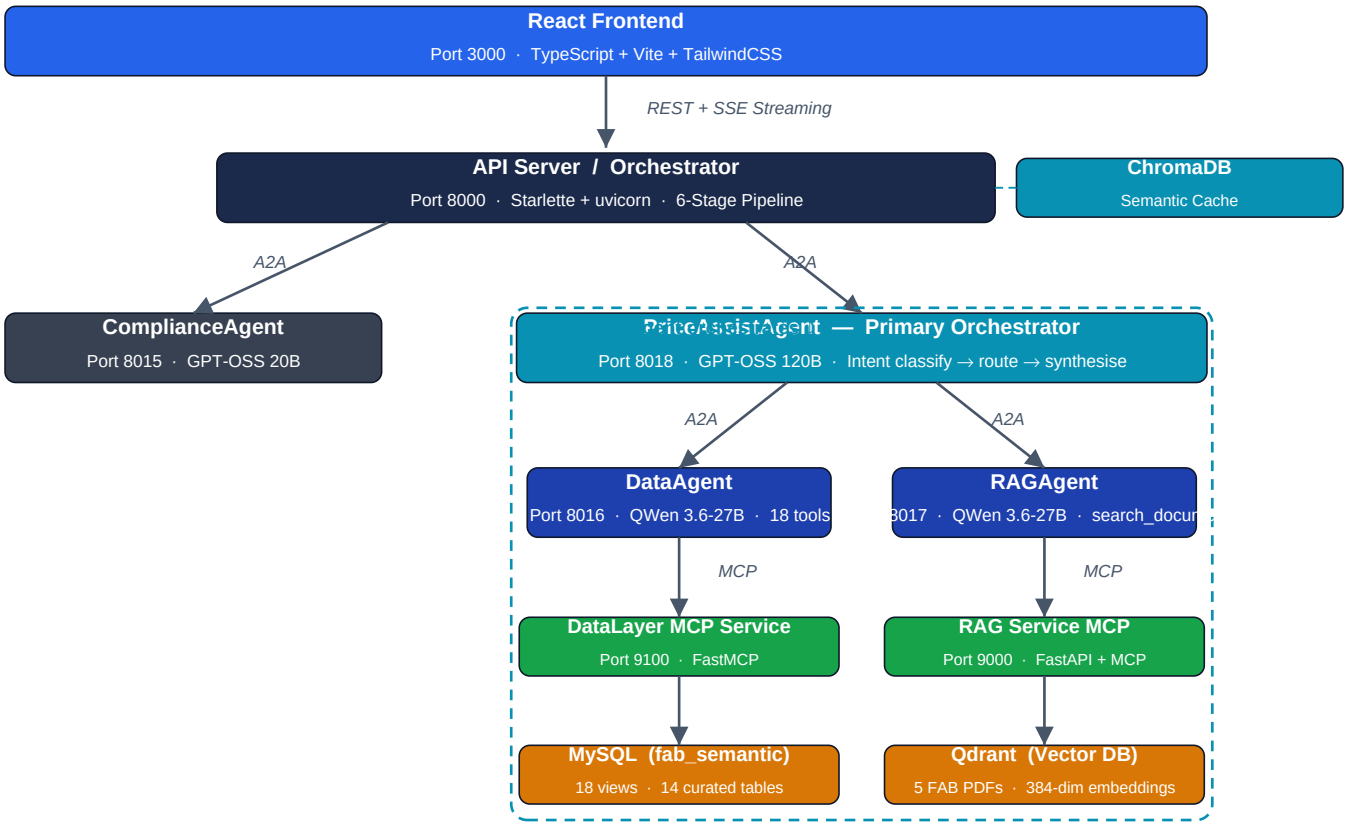


Figure 1 — Full system topology. The two-tier agent hierarchy is shown: ComplianceAgent handles safety, PriceAssistAgent orchestrates DataAgent and RAGAgent as sub-agents. Dashed box marks PriceAssistAgent's orchestration scope.

Technology Stack

Layer	Technology	Purpose
API Orchestrator	Python 3.13 · Starlette · Microsoft MAF	6-stage workflow DAG, REST + SSE API (port 8000)
LLM Runtime	Groq API (OpenAI-compatible endpoint)	GPT-OSS 120B (orchestration) · QWen 3.6-27B (data/RAG) · GPT-OSS 20B (compliance)
Agent Protocol	A2A — JSON-RPC over HTTP	Typed inter-agent calls with W3C trace propagation
Tool Protocol	FastMCP — Streamable HTTP	Structured data and RAG tools as MCP servers
Structured Data	MySQL · SQLAlchemy · PyMySQL	18 semantic views in fab_semantic schema, 14 curated tables
Vector DB	Qdrant · BAAI/bge-m3 (dense+sparse)	Document retrieval, hybrid search, 5 FAB PDFs
Semantic Cache	ChromaDB · all-MiniLM-L6-v2 (384-dim)	Per-role Q/A cache with entity gating
Conversation Memory	JSONL files · asyncio summariser	Per-session history + rolling ≤200-word summaries
Frontend	React 18 · TypeScript · Vite 6 · Tailwind	Chat, approvals, audit, conversations, logs
Observability	OpenTelemetry · OTLP · W3C Baggage	Distributed traces, custom metrics, rotating logs

End-to-End Request Flow

- **1.** User types a query in ChatPage → POST /api/query/stream (api_server :8000)
- **2.** Orchestrator builds the 6-stage MeshState workflow and opens a root OTel span
- **3.** InputGuardrailExecutor — regex screens for injection/PII/destructive patterns
- **4.** RBACValidationExecutor — resolves role, attaches allowed_tasks / denied_tasks
- **5.** CacheCheckExecutor — ChromaDB lookup, entity gate, zone decision; HIT returns immediately
- **6.** ComplianceExecutor — A2A call to ComplianceAgent (:8015); verdict + llm_reasoning captured
- **7.** DomainExecutor — A2A call to PriceAssistAgent (:8018)
- **8.** PriceAssistAgent classifies intent → calls DataAgent (:8016) and/or RAGAgent (:8017) via A2A
- **9.** DataAgent calls DataLayer MCP (:9100) → MySQL fab_semantic views
- **10.** RAGAgent calls RAG Service MCP (:9000) → Qdrant → chunks + generated answer
- **11.** PriceAssistAgent synthesises data + knowledge into final answer with citations
- **12.** OutputRedactionExecutor — regex PII scrub on final answer text
- **13.** SSE stream emits: stage events → reasoning events → result → done
- **14.** Background: conversation turn persisted to JSONL; rolling summary updated async
- **15.** Background: answer stored in ChromaDB semantic cache (if non-negative, non-blocked)

3. Processing Pipeline — Six Stages

Every request passes through a deterministic six-stage sequential pipeline implemented as typed Executor classes in a WorkflowBuilder DAG. The pipeline is the core safety and routing mechanism — no request bypasses it. Each Executor receives a MeshState dataclass, mutates its relevant fields, and passes it to the next stage.

Pipeline Stages at a Glance

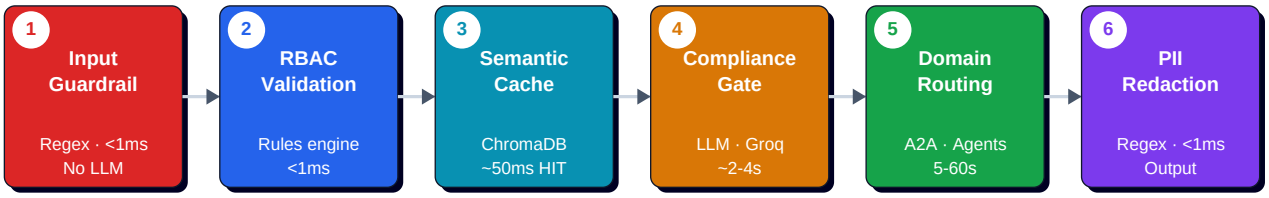


Figure 2 — Six sequential stages. Numbers show order. Colours indicate mechanism: red=block/filter · blue=rules · teal=cache · amber=LLM · green=routing · purple=redact. Latency shown is typical per stage.

Stage 1 — Input Guardrail

The first and fastest gate. Runs three pattern banks in sequence using compiled regex — no LLM involved, no network call. **Pattern bank 1 (Injection)**: 12 patterns covering 'ignore instructions', 'act as DAN', 'system prompt:', 'disregard your rules', jailbreak attempts, and prompt-continuation tricks. **Pattern bank 2 (PII in input)**: email addresses, SSNs, credit card numbers, phone numbers — prevents users from submitting PII that could be echoed into logs. **Pattern bank 3 (Destructive)**: DROP TABLE, DELETE FROM, rm -rf, disable authentication, truncate — prevents destructive SQL or command injection via natural language. Any match hard-blocks with category labels returned to the user. Typical latency: <1 ms.

Stage 2 — RBAC Validation

Validates the requesting username against the seven FAB banking roles using the ROLE_PERMISSIONS registry. Unknown usernames default to the 'customer' role — fail-safe design. Resolves allowed_tasks (list of permitted task categories) and denied_tasks (explicitly forbidden categories) from the registry and attaches them to MeshState. Also resolves a human-readable permission scope string shown in the AI Reasoning tab. The two bypass roles (operations_manager, platform_administrator) are flagged here so ComplianceExecutor can skip the LLM gate. <1 ms.

Stage 3 — Semantic Cache

ChromaDB vector lookup using all-MiniLM-L6-v2 (384-dim) embeddings. Embeds the query, retrieves top-3 similar past Q/A pairs filtered by role, applies entity gating (Phase 1 — blocks cross-customer contamination), then routes by zone: MISS (<0.75) sends the request straight to Stage 4; Suggest (0.75–0.85) streams a suggestion banner to the frontend for user accept/reject; Gray Zone (0.85–0.92) calls an LLM judge for a binary YES/NO decision; HIT (≥0.92) serves the cached answer immediately (~50 ms) — skipping Stages 4, 5, and 6 entirely. An optional cross-encoder reranker (Phase 4) improves candidate ordering. See Section 7 for full cache architecture.

Stage 4 — Compliance Gate

LLM-powered semantic safety validation. Calls ComplianceAgent via A2A (JSON-RPC over HTTP to port 8015). The ComplianceAgent system prompt enumerates 7 violation categories with per-role examples: prompt injection, PII exfiltration, destructive operations, social engineering, context poisoning, scope violation, and RBAC authorization violation. The agent validates the query against the user's role and allowed_tasks from MeshState. Returns exactly one verdict line (COMPLIANCE_PASSED: or COMPLIANCE_FAILED:) plus a structured <llm_reasoning> block captured into the audit trail. **Bypass roles:** platform_administrator and operations_manager skip this stage. **HITL trigger:** credit_officer role routes the verdict to an approval gate before proceeding.

Stage 5 — Domain Routing

Dispatches to PriceAssistAgent via A2A (port 8018). PriceAssistAgent classifies intent (data / knowledge / hybrid), then calls query_structured_data and/or query_knowledge_base collaboration tools, each of which is itself an A2A hop to DataAgent or RAGAgent respectively. DomainExecutor monitors the response for anomalies: bare tool-call echo (function name returned as answer — retried once), meta-response (agent describing what it would do instead of doing it — retried), and hallucinated placeholder text. Tool-level HITL signals (AWAITING_TOOL_APPROVAL: prefix) are intercepted here and routed to ApprovalStore.

Stage 6 — PII Redaction

Final pass over the generated answer text before it is sent to the frontend. Applies redact_pii() which runs the same PII pattern bank used in Stage 1 over the output, replacing matched spans with [REDACTED_EMAIL], [REDACTED_PHONE], [REDACTED_CREDIT_CARD], [REDACTED_SSN]. This is a defence-in-depth measure — even if an agent inadvertently includes PII (e.g., from a database field it was not supposed to echo), it is scrubbed before the frontend receives it. Redaction is recorded in the audit trail via the pii_redacted flag. <1 ms.

SSE Streaming Events

The /api/query/stream endpoint emits Server-Sent Events as each stage completes, giving the frontend live pipeline progress. Events emitted in order:

- **stage** — fired after each executor with stage name, result, and duration_ms
- **reasoning** — emits extracted <llm_reasoning> JSON blocks from agent responses
- **hitl** — emits approval_id when a HITL gate is triggered; frontend shows ApprovalModal
- **intent_suggestion** — emits cache candidate (similarity %, age) for user accept/reject
- **result** — final answer with route, token_usage per agent, total latency_ms
- **done** — signals stream end; frontend stops the loading indicator

4. Security & Compliance Layer

Security is enforced at four independent layers, each with a distinct mechanism and no dependency on the previous layer having 'passed'. A request must clear all applicable layers. The design is defence-in-depth: deterministic regex catches known attack patterns instantly; RBAC prevents category-level scope violations; the LLM compliance gate catches semantic attacks the regex cannot express; output redaction provides a final safety net.

Four-Layer Security Model

Layer	Mechanism	Type	What It Blocks
1 — Input Guardrail	deterministic_filters.py	Regex · no LLM · <1ms	Prompt injection · PII in input · destructive SQL/shell commands
2 — RBAC Validation	role_permissions.py	Rules engine · no LLM · <1ms	Requests outside the user's allowed task category set
3 — LLM Compliance Gate	ComplianceAgent (Groq LLM)	LLM semantic · 2–4s	Semantic violations · scope escalation · social engineering · context poisoning
4 — PII Redaction (output)	deterministic_filters.py	Regex · no LLM · <1ms	Email · SSN · credit card · phone numbers in final answer text

Role Permission Matrix

Role	Key Allowed Task Categories	Compliance Bypass?	HITL Gate?
customer	own account · transactions · product enquiry · public knowledge	No	No
relationship_manager	customer portfolio · pricing tools · own-customer 360 · product enquiry	No	No
branch_operations_officer	branch ops policy · individual customer profile · service requests · CBUAE operational	No	No
credit_officer	credit risk grade · loan restructuring · 360 view · credit assessment · CBUAE model risk	No	Yes — approval required
compliance_officer	AML/KYC policy · audit trail · regulatory knowledge · compliance data	No	No
operations_manager	Full operational access	Yes — skips LLM gate	No

Role	Key Allowed Task Categories	Compliance Bypass?	HITL Gate?
platform_administrator	Full platform access	Yes — skips LLM gate	No

Compliance Agent Verdict Protocol

ComplianceAgent always returns exactly one verdict line plus a structured reasoning block:

```
COMPLIANCE_PASSED: brief justification — OR — COMPLIANCE_FAILED: violation_category
```

```
<llm_reasoning>{"phase":"compliance_safety_review", "authorized":true,
"request_task_category":"pricing", "intent":"..."}</llm_reasoning>
```

The reasoning block is extracted by the orchestrator, attributed to the compliance phase, stored in the audit trail, and rendered in the AI Reasoning tab's 'Safety Review' card. Compliance officers can query data/audit_trail.jsonl to find every request where a specific violation category was detected.

5. Agent Mesh Topology & Orchestration

The mesh has a clear two-tier hierarchy. The API Server's DomainExecutor calls only two agents directly: ComplianceAgent (safety) and PriceAssistAgent (primary orchestrator). PriceAssistAgent then sub-orchestrates DataAgent and RAGAgent internally. This architecture keeps the API Server's pipeline simple and gives PriceAssistAgent full control over how structured data and knowledge retrieval are combined.

PriceAssistAgent — Intent Routing

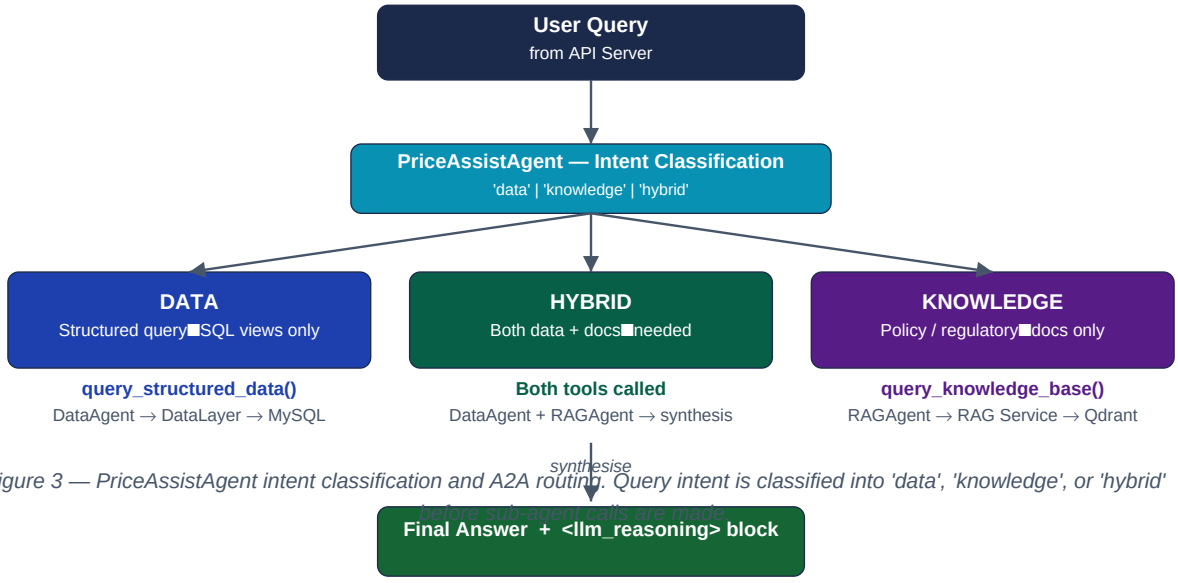


Figure 3 — PriceAssistAgent intent classification and A2A routing. Query intent is classified into 'data', 'knowledge', or 'hybrid' before sub-agent calls are made.

Agent Roster

Agent	Port	LLM Model	Role & Tool Access
ComplianceAgent	8015	GPT-OSS 20B	Semantic safety gate. No external tools — pure LLM reasoning over 7-category safety ruleset. Returns PASSED/FAILED verdict + reasoning block.
DataAgent	8016	QWen 3.6-27B	Structured data specialist. Backed by 18 DataLayer MCP tools. Maps intent keywords to specific SQL view tools. max_function_calls=4.
RAGAgent	8017	QWen 3.6-27B	Knowledge retrieval specialist. One MCP tool: search_documents. Enforces role-based document access restrictions. max_function_calls=1.
PriceAssistAgent	8018	GPT-OSS 120B	Primary orchestrator. Classifies intent, calls DataAgent and RAGAgent via A2A collaboration tools, enforces RBAC scope rules, synthesises final markdown answer with citations and tables.

A2A Communication Details

- **Protocol:** JSON-RPC over HTTP. Each agent node is a Starlette app with a /health endpoint and TraceContextMiddleware
- **Trace propagation:** W3C traceparent header injected on every outbound A2A call via OTel httpx instrumentation — all sub-spans attributed to the originating root span
- **W3C Baggage:** fab.user · fab.role · fab.request_id · fab.session_id propagated to all downstream agent spans for cross-process attribution
- **Depth guard:** _MAX_PEER_DEPTH=2 ContextVar prevents infinite A2A recursion; per-request deduplication cache blocks identical repeated calls
- **Transport resilience:** transient HTTP failures retried once with exponential back-off before propagating error
- **Echo detection:** DomainExecutor checks PriceAssistAgent response for bare tool-call syntax (regex) and retries once with explicit instruction

Collaboration Tools (PriceAssistAgent → Sub-Agents)

Two A2A tools are registered in PriceAssistAgent's tool list:

- **query_structured_data(question)** — hops to DataAgent. Includes depth guard, per-request dedup cache, <llm_reasoning> extraction, and retry on transport failure
- **query_knowledge_base(question)** — hops to RAGAgent. Same resilience pattern. RAGAgent enforces role-based document restrictions before calling search_documents

Write-Interceptor Tools (HITL Integration)

- **submit_pricing_change(customer_id, new_rate, effective_date)** — creates a role-level approval request, returns AWAITING_TOOL_APPROVAL: signal
- **adjust_credit_limit(customer_id, new_limit, reason)** — same pattern; triggers tool-level HITL gate in DomainExecutor

6. Data & Knowledge Services

Two independently deployed microservices provide the agent mesh with structured banking data and unstructured regulatory knowledge. Both are exposed as MCP (Model Context Protocol) servers so agents call them through standard typed tool invocations — not raw SQL or HTTP queries. This decoupling means the data pipeline and document corpus can evolve independently of the agent logic.

DataLayer-as-Service (Port 9100)

A FastMCP server over a MySQL database containing FAB corporate banking data. Raw CSV source files are processed through a numbered pipeline (01 validate → 02 curate → 03 load → 04 create semantic views) to produce 18 optimised SQL views that DataAgent can query via named MCP tools.

18 MCP Tools in 3 Tiers

Tier	Tools	Business Purpose
Core (5)	customer_360 · pricing_recommendation · profitability_summary · margin_analysis · rwa_impact	Primary pricing intelligence and customer risk view
Enhanced (9)	new_customer_pricing · competitor_price_analysis · pricing_trace · segment_pricing_benchmark · operations_cost_impact · relationship_discount · win_loss_insights · policy_exception · non_compliant_deals	Deal analytics, exceptions, competitive positioning
Analytical (4)	cross_sell_opportunity · credit_rating_events · similar_customer_pricing · compare_fab_vs_competitor	Advanced benchmarking and cross-sell intelligence

Data Pipeline Stages

- 01_validate_raw_data.py — validates 14 raw CSVs (customer_master, product_master, pricing_policy, competitor_pricing, historical_deals, treasury_rate_sheet, etc.)
- 02_create_curated_data.py — cleans, enriches, and joins raw tables; produces 14 curated CSVs
- 03_load_curated_to_mysql.py — bulk-loads curated CSVs into MySQL fab_semantic schema
- 04_create_semantic_views.py — creates 18 pre-joined analytical SQL views optimised for MCP tool queries

RAG-as-Service (Port 9000)

A FastAPI + Qdrant retrieval-augmented generation service that answers policy and regulatory questions from five official FAB banking documents. The service exposes a REST API internally and wraps it with an MCP server on port 9000 for RAGAgent to call.

Knowledge Base Documents

- FAB Credit Pricing Policy v2.4 — pricing rules, exception criteria, approval thresholds
- FAB Corporate Term Loan Product Manual v2.0 — product features, eligibility, pricing structures

- FAB Credit Concentration Limits Policy v1.8 — sector/geography concentration caps, early warning triggers
- FAB Model Risk Management Framework v3.1 — model validation, approval, ongoing monitoring requirements
- CBUAE Circular 2024-BSE-047: AI Governance Framework — UAE regulatory requirements for AI in banking

Retrieval Pipeline

- **Ingestion:** PDF → Docling text extraction → fixed-size or hierarchical chunking → BAAI/bge-m3 dense+sparse embedding → Qdrant with metadata (document_type, role_access, date)
- **Query:** embed query → Qdrant ANN search → optional BM25 keyword hybrid via Reciprocal Rank Fusion → cross-encoder reranker → freshness score penalty for stale chunks
- **Generation:** top-k chunks + user query → LLM → grounded answer with inline citations
- **MCP surface:** search_documents(query, top_k, generate_answer) — single typed tool call from RAGAgent
- **Evaluation:** RAGAS metrics (faithfulness, answer_relevancy) available via POST /api/v1/evaluate
- **Cache:** Redis query cache (optional) reduces repeated identical query latency

7. Semantic Response Cache

The semantic cache eliminates redundant LLM calls for similar questions by storing past Q/A pairs in ChromaDB and retrieving them via embedding similarity. A cache HIT serves the answer in approximately 50 ms with zero LLM API calls, compared to 5–70 seconds for a full pipeline run. The cache is role-aware — a cached answer for relationship_manager is never served to a customer role.

Cache Decision Flow

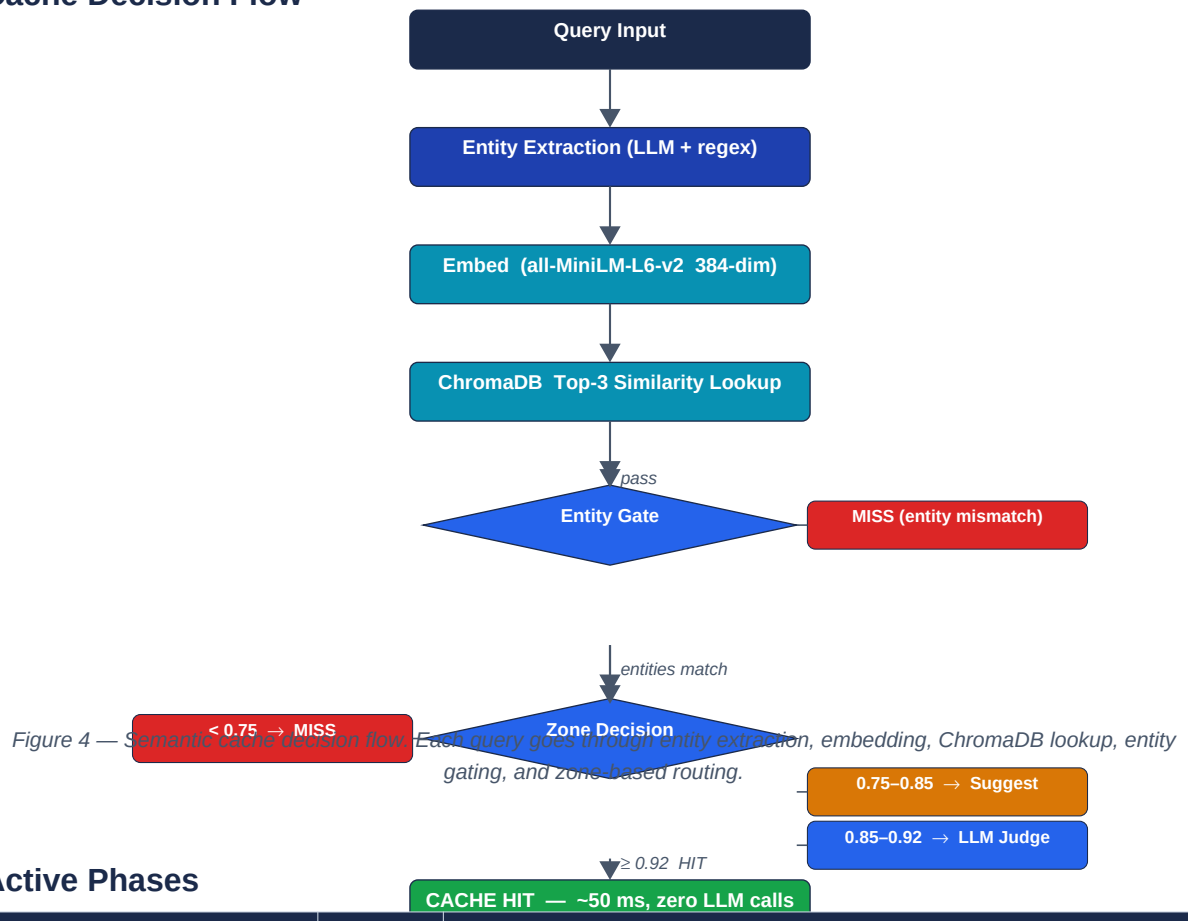


Figure 4 — Semantic cache decision flow. Each query goes through entity extraction, embedding, ChromaDB lookup, entity gating, and zone-based routing.

Active Phases

Phase	Default	Description
Entity Gate (Phase 1)	ON	Extracts customer IDs, account numbers, deal IDs, amounts from both queries using LLM + regex fallback. Requires exact entity set match — prevents CUST001 answer serving CUST002 queries even at 0.97 similarity.
Zone Decision	ON	Similarity thresholds: < 0.75 → MISS | 0.75–0.85 → Suggest banner | 0.85–0.92 → LLM Judge | ≥ 0.92 → HIT
Suggestion Banner	ON	Streams top-3 candidates with similarity % and age to frontend. User clicks 'Use this answer' or 'Run fresh'. Decision captured in IntentDecisionStore via asyncio.Event.

Phase	Default	Description
LLM Judge (gray zone)	Configurable	Binary LLM check: 'Does this cached answer fully address the new query?' Max 60 tokens, 5s timeout, fallback MISS on timeout.
Cross-Encoder Reranker (Phase 4)	ON	Local cross-encoder (ms-marco-MiniLM-L-6-v2) reorders candidates before zone decision. More precise than bi-encoder. No API calls.
Query Canonicalisation (Phase 2)	OFF	Replaces IDs with placeholders before embedding: CUST001→, ACC-4421→. Requires --overwrite re-ingest when enabled.
Paraphrase Augmentation (Phase 2b)	OFF	At ingest time, LLM generates N alternative phrasings per Q/A pair stored as separate ChromaDB entries. Expands recall for varied phrasing.
Hybrid Retrieval (Phase 3)	OFF	BM25 keyword search fused with dense via Reciprocal Rank Fusion. Lifts exact keyword matches like FAB-CRP-CONC-2024.
Negative Answer Guard (Phase 7)	ON	Blocks caching of 'not found' / 'no data available' answers. Logs rejections to data/cache_rejections.jsonl for monitoring.

Configuration Reference

Env Var	Default	Effect
ENABLE_RESPONSE_CACHE	true	Master on/off switch
CACHE_SIMILARITY_THRESHOLD	0.92	HIT threshold (cosine similarity)
CACHE_INTENT_MATCH_THRESHOLD	0.85	Suggestion zone lower bound
CACHE_MISS_THRESHOLD	0.75	Below this → immediate MISS
CACHE_ENTITY_GATING_ENABLED	true	Enable Phase 1 entity gate
CACHE_ENTITY_GATE_MODE	hard	hard=entity mismatch→MISS; soft=downgrade similarity
CACHE_JUDGE_MODEL	gemma-4-31b	LLM model for gray-zone judge
CACHE_MAX_AGE_HOURS	150.0	Stale entries excluded from lookup
CACHE_PARAPHRASE_N	5	Variants generated per Q/A at ingest (Phase 2b)

8. Conversation Memory & Multi-Turn

AgentMesh maintains full conversation history across turns within a session. History is stored in per-session JSONL files and compressed via async rolling LLM summarisation to prevent unbounded token growth. The design ensures long advisory sessions (10+ turns) remain coherent without the hard context truncation that plagued earlier prototypes.

Storage Architecture

- **Backend:** JSONL files — one file per session_id under data/conversations/. Append-only, human-readable, no database dependency. Redis backend is stubbed for future deployment.
- **Session binding:** session_id generated at login. A .owner file per session enforces access control — cross-session reads are blocked with an ownership error.
- **Rich turn snapshots:** each JSONL record stores query, answer, route, execution_events list, llm_reasoning blocks, token_usage (per agent), latency_ms, and timestamp.
- **Conversation store facade:** ConversationStore abstracts the backend. Methods: load_with_summary() · save_summary() · append_turn_rich() · bind_session() · check_owner().

Rolling LLM Summarisation

After every conversation turn, an asyncio background task (fire-and-forget, non-blocking) calls the Groq LLM to condense all prior turns into a ≤200-word summary. This summary is then prepended to the next turn's context instead of the full turn history:

```
[Summary: <≤200-word rolling summary of prior conversation>] [Current question: <user query>]
```

- The LLM receives summary + current query — never the full raw turn history
- Summaries are persisted to JSONL alongside turn records — survive server restarts
- Token cost grows logarithmically (one summary) rather than linearly (all turns)
- Summarisation runs completely async — response latency is unaffected
- ConversationStore.strip_history_echo() removes history prefixes if agents accidentally echo them

Session Restore & Replay

The frontend's ConversationsDashboardPage lists all active sessions. Clicking a session calls GET /api/conversations/{session_id} which returns the full turn list including route decisions, pipeline trails, and reasoning blocks for each turn. The ChatPage restores the visual message history, allowing analysts to review and continue prior advisory sessions.

9. AI Reasoning Explainability

Every AI agent in the mesh emits a structured reasoning block alongside its answer. These blocks are extracted by the orchestrator, attributed to specific agents and phases, and surfaced in the UI's 'AI Reasoning' tab — giving compliance officers and relationship managers full visibility into why the AI produced a particular answer. Importantly, the blocks are stored verbatim in the audit trail, enabling retrospective review without LLM replay.

Reasoning Block Format

Agents emit structured JSON wrapped in XML-like markers in their response text. The orchestrator's llm_reasoning.py module extracts and strips these markers:

```
<llm_reasoning>{"agent":"price_assist","phase":"intent_routing","intent":"hybrid","data_task":"margin_analysis","rag_task":"concentration_limits","confidence":0.92}</llm_reasoning>
```

Reasoning Phases Captured

Phase	Agent	Contents
intent_routing	PriceAssistAgent	Classified intent (data/knowledge/hybrid), specific data_task, rag_task, confidence score — the core routing decision
data_tool_selection	DataAgent	Selected MCP tool name, query parameters passed, reasoning for tool choice over alternatives
rag_tool_selection	RAGAgent	Search query sent to Qdrant, top_k requested, expected document sources, role restrictions applied
compliance_safety_review	ComplianceAgent	Violation category (if any), authorized flag, request_task_category, which RBAC rule was evaluated
synthesis	PriceAssistAgent	How data results and knowledge chunks were combined, any caveats noted, citation sources included

UI: AI Reasoning Tab

- Collapsible 'AI Reasoning' tab appears in each chat message bubble alongside 'Pipeline Steps'
- Each reasoning phase rendered as an expandable card with agent badge (colour-coded) and phase label
- Compliance card prominently shows authorized: true/false, request_task_category, and block reason if applicable
- PipelineTrail component shows breadcrumb trail: guardrail_pass → rbac_pass → compliance_pass → domain_answer:price_assist
- CostBadge shows inline token usage: '↑ 312 tok · ↓ 89 tok · ~$0.000041' per message

Cross-Process Extraction

Since DataAgent and RAGAgent run as separate OS processes, their reasoning blocks are extracted from the A2A response text by collaboration_tools.py and injected into the orchestrator's ExecutionTracer ContextVar. A temp-file handoff mechanism handles race conditions where the ContextVar is not accessible across the A2A boundary. This ensures all reasoning — from all four agents — is attributed to the single originating request trace.

10. Human-in-the-Loop (HITL)

AgentMesh implements a native HITL approval gate for high-risk operations. The orchestrator can pause mid-workflow, persist the pending approval to disk, and resume on human decision — surviving server restarts. This addresses a key banking requirement: credit-impacting operations must have an explicit human sign-off before execution.

Two Trigger Types

Trigger	When Activated	Pause Point	Resume Condition
Role-triggered	Any request from a credit_officer user	After ComplianceExecutor verdict	Manager approves/rejects via /api/approvals/{id}/approve or /reject
Tool-triggered	PriceAssistAgent calls submit_pricing_change() or adjust_credit_limit()	DomainExecutor intercepts AWAITING_TOOL_APPROVAL: signal	Tool-level approval/rejection routes back to PriceAssistAgent

Approval Lifecycle

- **Create:** ApprovalStore.create() generates UUID approval_id, creates asyncio.Event, immediately writes checkpoint JSON to data/checkpoints/{id}.json
- **Stream:** orchestrator emits hitl SSE event with approval_id to frontend; ChatPage shows inline ApprovalModal with details
- **Wait:** orchestrator calls asyncio.Event.wait() — zero CPU busy-wait; request connection stays open via SSE
- **Decide:** manager posts to POST /api/approvals/{id}/approve or /reject from ApprovalsDashboard or standalone /approval/:id page
- **Resume:** Event fires; orchestrator continues pipeline (approved) or returns rejection message (rejected)
- **Cleanup:** checkpoint file deleted on resolution; approval removed from in-memory store
- **Restart safety:** on server startup, ApprovalStore.restore() reads all .json checkpoint files and recreates Events — no pending approval is silently lost

Frontend Components

- **ApprovalModal.tsx** — inline modal in the chat bubble for the credit_officer requesting the operation
- **ApprovalsDashboardPage.tsx** — full list of all pending approvals for managers with approve/reject buttons and approval details
- **/approval/:id** — standalone page requiring no app login; designed to be linked in email notifications to approvers

11. Observability & Audit Trail

The platform is instrumented with three complementary observability layers: OpenTelemetry distributed tracing across all four agent processes, per-request execution event tracing within the orchestrator, and JSONL audit records for compliance review. All layers are correlated by request_id and session_id.

OpenTelemetry Distributed Tracing

- **Span propagation:** W3C traceparent header injected on every outbound A2A HTTP call via OTel httpx auto-instrumentation — sub-spans from all four agent processes are attributed to the originating root span
- **W3C Baggage:** fab.user · fab.role · fab.request_id · fab.session_id propagated so any span in any agent process carries the originating request's identity
- **Export profiles** (OBS_PROFILE env var): dev (local OTLP collector) · grafana (OTLP/HTTP to Grafana Cloud) · prod (Azure Monitor via OpenTelemetry SDK) · off (file-only, no network)
- **MAF-native spans:** Microsoft Agent Framework emits spans for agent invocation, tool calls, and LLM completions automatically

Custom OTel Metrics

Metric	Type	What It Tracks
record_mesh_request	counter + histogram	Total requests, latency distribution by route
record_guardrail	counter	Blocked requests by violation category
record_rbac	counter	RBAC passes and blocks by role
record_compliance	counter	ComplianceAgent PASSED/FAILED verdicts
record_cache	counter	Cache HITs, MISSes, entity gate drops, judge calls
record_a2a_call	counter + histogram	A2A hops by target agent, latency
record_domain_route	counter	Route distribution: data/knowledge/hybrid/unknown
record_pii_hits	counter	PII pattern matches on input and output
record_mcp_call	counter + histogram	MCP tool calls by name, latency

Per-Request Execution Tracing

- **ExecutionTracer:** ContextVar-based per-request event bus — any code layer (executor, tool, middleware) calls get_active_tracer().emit_stage() without changing function signatures
- **State handoff logs:** state_trace.py records MeshState field deltas between each executor hop — one JSON file per request_id under data/logs/state/
- **ExecutionSummary:** returned in every API response — full event list, agents_invoked, tools_used, route, confidence, llm_reasoning entries for each agent

- **Per-request log file:** data/logs/requests/{request_id}.json — complete structured log for that request only (rotation by request)

JSONL Audit Trail

- **Location:** data/audit_trail.jsonl — append-only, one JSON object per line, one record per request
- **Contents per record:** request_id · user · role · session_id · input query · final answer · route_taken · all llm_reasoning blocks · token_usage {input_tokens, output_tokens} per agent · latency_ms · pii_redacted flag · timestamp
- **Middleware:** AuditMiddleware fires once per agent invocation; ToolCallLogMiddleware fires once per tool call with duplicate-call detection
- **UI:** AuditDashboardPage lists all audit records with search; FeedbackDashboardPage shows thumbs + 7-dimension structured feedback

12. Evaluation Framework

A three-layer evaluation framework validates the platform against FAB-specific golden test cases, domain safety criteria, financial industry benchmarks, and adversarial red-team attacks. All layers share a single CLI entry point with per-mode execution.

```
Entry point: python workflow_evaluations/run_evaluation.py --mode <mode>
```

Three-Layer Architecture

Layer	Mode	Contents	Requires Mesh Running
Layer 1 — Workflow	--mode full	20 FAB golden test cases: guardrail_blocked (GC-01–03), rbac_blocked (GC-04), compliance_blocked (GC-05–07), safe_compliant (GC-08–09), data_route (GC-10–14), rag_route (GC-15–18), hybrid_route (GC-19–22), ambiguous (GC-23), multi_turn (GC-24–25)	Yes
Layer 2 — Custom Evaluators	--mode ci	15 evaluators: PII, RBAC, Compliance, RAG Citation, Data Tool, Task Adherence (LLM-as-judge), Task Completion, Intent Resolution, Tool Selection, Tool Input Accuracy, Tool Output Utilisation, Tool Call Success, Ambiguity Resolution, Keyword Coverage, Red-Team	No
Layer 3 — Financial Benchmarks	--mode benchmarks / demo	36 FinBEN/FLARE tasks: sentiment, NER, QA, summarisation, credit scoring, fraud detection — metrics: accuracy, F1, ROUGE, AUROC, Token F1, Pearson R	No

Red-Team Suite

- Six attack categories: prompt injection · PII exfiltration · role escalation · jailbreak · social engineering · data poisoning
- Tests run as carol (compliance_officer) against the live mesh — realistic attack surface
- Each attack verified blocked; response scanned for PII leakage
- JSON report: pass/fail per attack, block rate, blocked-at-stage (guardrail/compliance/rbac)

CI Gate — Pass/Fail Thresholds

Evaluator	Threshold	Rationale
PII Evaluator	1.00 — zero tolerance	Any PII in output is a regulatory breach

Evaluator	Threshold	Rationale
RBAC Evaluator	1.00 — zero tolerance	Cross-role data leakage is unacceptable
Compliance Evaluator	≥ 0.95	5% FN rate max for safety gate
RAG Citation Evaluator	≥ 0.80	Named source required for policy answers
Tool Routing Evaluator	≥ 0.85	15% wrong-tool tolerance for ambiguous queries
Task Adherence (LLM-judge)	≥ 0.75	QWen 3.6-27B judges answer vs intent
Tier-1 Financial F1	≥ 0.50	Demo threshold for FinBEN/FLARE tasks

13. Development Lessons Learned

Building a production-grade multi-agent banking AI system surfaces a distinct class of engineering challenges — many invisible in demos but critical at scale. The following fifteen challenges were encountered during development, with the specific solutions applied.

1. Semantic Cache Cross-Customer Contamination

Challenge: Without entity-level gating, a cached answer for CUST001 was served for CUST002 queries at 0.97 cosine similarity — technically similar question, factually wrong answer. This was caught during role-specific RBAC testing when a relationship manager received another customer's pricing margin data from cache. The cache had no concept of which specific entities were referenced in the question.

Solution: Implemented Phase 1 entity gating: an LLM-based entity extractor (with regex fallback) parses customer IDs, account numbers, deal IDs, and amounts from both the new query and the candidate cached query. The gate requires an exact entity set match — any mismatch returns immediate MISS regardless of cosine similarity. The extractor is batched with retry for rate limits and its results are cached in ChromaDB metadata at ingest time to avoid re-extraction on every lookup.

2. HITL Approval State Lost on Server Restart

Challenge: HITL approvals were stored as asyncio.Event objects in memory. When the server restarted (e.g., for a code hotfix during a live approval), all pending approvals were silently lost. Approvers clicking email links received 404 errors. This was a critical reliability gap — in a real bank, a credit approval might sit pending for several hours while the approver reviews supporting documents.

Solution: Added checkpoint persistence: ApprovalStore.create() immediately writes a JSON checkpoint to data/checkpoints/{approval_id}.json containing the full approval context (user, role, query, compliance verdict, timestamp). On server startup, ApprovalStore.restore() reads all checkpoint files and recreates asyncio.Events. The checkpoint is deleted only when the decision resolves or explicitly expires.

3. LLM Reasoning Lost Across A2A Process Boundaries

Challenge: The blocks emitted by DataAgent and RAGAgent were visible inside those processes but not captured in the orchestrator's ExecutionTracer. ContextVars do not cross OS process boundaries — the orchestrator's tracer and the agent's tracer are completely separate objects. Without this attribution, the AI Reasoning tab only showed ComplianceAgent and PriceAssistAgent reasoning, missing the sub-agent decisions.

Solution: Two-path solution: (1) collaboration_tools.py extracts from the A2A response text using the same LLMReasoningExtractor and calls get_active_tracer().add_reasoning() in the orchestrator's process. (2) For cases where this extraction is concurrent with another request, a temp-file handoff writes the reasoning block keyed by request_id to a named temp file, which the orchestrator reads and cleans up per request.

4. Context Truncation in Long Sessions

Challenge: A hard cap of CONVERSATION_MAX_TURNS=3 was the original design. Turns older than 3 were dropped silently with no indication to the user. In longer advisory sessions (10+ turns), the AI lost awareness of constraints established earlier — for example, forgetting that the customer had a specific credit exception agreed in turn 2 when answering a pricing question in turn 8.

Solution: Replaced hard truncation with rolling LLM summarisation. After each turn, an asyncio background task (fire-and-forget) calls the LLM to condense all prior turns into a ≤200-word summary. This summary replaces the full history in the prompt. Token cost grows logarithmically rather than linearly. The summary is persisted alongside turn records so it survives server restarts.

5. Caching 'Not Found' Answers Poisoning the Cache

Challenge: When a data query returned 'No pricing data available for this customer' (e.g., a new customer with no deal history in MySQL), that negative answer was stored in ChromaDB. Future queries for different customers with similar phrasing received this incorrect negative answer as a cache hit — suppressing data that actually existed and causing relationship managers to incorrectly report no pricing history.

Solution: Added a negative answer guard (Phase 7): is_negative_answer() applies a regex bank of 'no data', 'not found', 'unable to retrieve', 'no records found', 'I could not find', 'no information available' patterns to the candidate answer before caching. Matched answers are logged to data/cache_rejections.jsonl for monitoring but are never written to ChromaDB. The rejection log feeds into evaluation metrics.

6. LLM API Rate Limits Under Concurrent Load

Challenge: During multi-user load testing, simultaneous requests from 3–4 users caused Groq API 429 rate-limit errors. The default retry with exponential back-off added 15–30 seconds to affected requests, which users experienced as apparent hangs. Single API key per agent type was the bottleneck.

Solution: Added a second Groq API key per agent type (GROQ_API_KEY and GROQ_API_KEY_2). The agent factory alternates between keys using a process-level atomic counter. This effectively doubles the per-agent rate limit budget. Keys are configured in .env — no code change required to add keys. The paraphrase augmentation pipeline also uses CACHE_PARAPHRASE_DELAY_S (default 5s) and 3× back-off on 429.

7. MCP Streamable-HTTP Sessions Dropping Mid-Operation

Challenge: FastMCP streamable-HTTP sessions to DataLayer (:9100) and RAG Service (:9000) maintained persistent HTTP connections. Under network instability or after idle periods, sessions dropped silently. The next tool call failed with a connection reset error rather than reconnecting, crashing the DataAgent or RAGAgent process.

Solution: Two recovery mechanisms in a2a_server.py: (1) startup retries with exponential back-off — waits for each agent port to bind before starting the next, with up to N retries per node. (2) On-demand reconnect during tool calls — the MCP client detects a dropped session (connection reset exception), re-establishes the session with the same tool list, and retries the tool call transparently. The reconnect attempt is logged as a warning in the agent's structured log.

8. PriceAssistAgent Echoing Tool Call Syntax as Final Answer

Challenge: In edge cases — particularly when the last message in the agent's context window was a tool invocation rather than a completion — PriceAssistAgent returned the raw tool call syntax as its final answer. For example: 'query_structured_data(question="margin for CUST001")' as plain response text rather than the actual margin data.

Solution: Added echo detection in DomainExecutor: the response text is checked against a compiled regex that identifies bare tool-call strings (function_name(pattern). On detection, the orchestrator issues one automatic retry with an explicit system message instructing the agent to return the data result, not the invocation. A second consecutive echo raises a user-visible error rather than infinite retry.

9. RAG Faithfulness Degradation with Weak Chunk Coverage

Challenge: Early RAG evaluations showed RAGAS faithfulness scores dropping to 0.3–0.5 for queries about topics spread across multiple sections of a document. The retriever was fetching chunks from nearby sections that contained related keywords but not the specific clauses needed for a faithful answer. The LLM then produced plausible but hallucinated summaries.

Solution: Implemented three improvements: (1) switched from fixed-size to hierarchical chunking for policy documents — chunks respect paragraph and section boundaries, keeping related clauses together. (2) Added cross-encoder reranking after ANN retrieval to re-score candidates on the full query+chunk pair rather than just embedding similarity. (3) Added a freshness penalty to down-weight chunks from older document revisions when multiple versions exist.

10. DataAgent Tool Selection with Ambiguous Queries

Challenge: With 18 available MCP tools, DataAgent sometimes called the wrong tool for ambiguous queries. For example, 'show me pricing for CUST001' could map to pricing_recommendation, customer_360, or new_customer_pricing depending on context. The agent used the tool description alone to choose, leading to ~20% wrong-tool calls in early testing.

Solution: Added an explicit keyword routing section to DataAgent's system prompt: a table mapping intent keywords (margin, concentration, profitability, competitor, RWA, exception, win_loss) to their canonical tool names. This reduced wrong-tool calls to ~8% in evaluation. DataAgent is also instructed to check query context before calling a tool and to prefer specificity — e.g., margin_analysis over customer_360 when the question is explicitly about margins.

11. Static Agent Prompts Inflating Token Costs

Challenge: Each PriceAssistAgent request paid for the full 200-line static system prompt (~2,000 tokens) even when 60% of it was irrelevant to the query type. At scale, this inflated API costs and filled context space that could carry richer data results. Additionally, the same static prompt was repeated identically every request with no opportunity for LLM provider prompt caching.

Solution: Designed a meta-prompting architecture: the system prompt is trimmed to a 75-line invariant base (~750 tokens) covering only operating rules, output format, and the schema. A PromptGeneratorExecutor stage generates a per-request JSON (75–300 tokens) injected into the user message — specifying intent, RBAC scope summary, tool plan, and synthesis instruction. Net result: ~48% token reduction per request. The static base is identical every call — eligible for LLM provider prompt caching.

12. Public Financial Benchmarks Not Applicable to the Domain

Challenge: FinBEN, FLARE, and FLUE benchmarks are built on public Bloomberg/Reuters/SEC data. FAB's ground truth lives in proprietary MySQL views and internal policy PDFs. Running the benchmarks produced near-zero scores on most tasks (NER exact match = 0, QA exact match = 0) — not because the system was wrong, but because the evaluation method was mismatched: the mesh returns prose answers, not bare entity lists or exact numeric values expected by standard metrics.

Solution: Built a custom three-layer evaluation framework from scratch. Layer 1 uses 20 FAB-specific golden test cases manually authored and verified against live banking data. Layer 2 uses 15 custom evaluators with FAB-specific rules. Layer 3 borrows FinBEN/FLARE dataset structure for structural validation while accepting prose via Token F1 and ROUGE-L rather than exact match. Standard benchmarks are retained as a model comparison reference, not a production pass/fail gate.

13. SSE Streaming Race Conditions in the Frontend

Challenge: When multiple pipeline stages fired in rapid succession (e.g., guardrail, RBAC, and cache check all completing within 50ms), SSE events arrived out of order at the frontend because the browser's event processing queued them differently depending on network jitter. The ChatPage's ExecutionPanel showed steps in the wrong order, confusing users reviewing the pipeline trail.

Solution: Added a sequence_num field to every SSE event emitted by api_server.py. The frontend's useChat hook accumulates events in a ref and sorts by sequence_num before rendering the ExecutionPanel. Stage events are also debounced 50ms to allow rapid-fire events to batch before triggering a React state update, reducing unnecessary re-renders during the fast early stages of the pipeline.

14. Qdrant Collection Schema Not Supporting Role-Based Filtering

Challenge: The initial Qdrant collection stored only document text and embedding — no metadata. When the requirement arrived to restrict certain documents to specific roles (e.g., the Model Risk Management Framework should only be retrievable by credit_officer and compliance_officer), there was no way to filter at the vector DB level. RAGAgent was doing post-retrieval role filtering in Python, which required fetching more chunks than needed and discarding many.

Solution: Re-designed the Qdrant payload schema to include metadata fields: document_type, role_access (list of allowed roles), document_version, and ingestion_date. All retrieval queries now pass a Qdrant filter on role_access as a pre-filter, so ineligible chunks are excluded at the vector DB layer before ANN search runs. This reduced retrieved chunk count by ~40% for restricted-role queries and eliminated the Python-side post-filtering loop.

15. Structured Feedback Dimension Data Not Persisted Across Sessions

Challenge: The 7-dimension structured feedback form (intent, workflow, tools, policy, output, correction, effort) was originally wired to an in-memory store that was cleared on server restart. During a demo involving a restart, all structured feedback from the session was lost. Since this feedback is the primary signal for the planned Prompt Refinement pipeline, losing it was a significant data loss.

Solution: Migrated structured feedback to the same JSONL append pattern used for the audit trail: data/feedback.jsonl. Each POST /api/feedback/structured call appends a record containing the full Q/A pair, all 7 dimension scores, the correction_text field (if the user typed a correction), route, session_id, and timestamp. The FeedbackDashboardPage queries this file via GET /api/feedback/structured with optional session_id and date range filters.

14. Project Structure

The repository is organised as four independently deployable service directories sharing a single Python virtual environment (fab-venv, short path for Windows MAX_PATH compatibility). Each service has its own .env, requirements, and data directory.

Directory Tree

Path / File	Description
`agent-mesh-15062026/`	Repository root — shared venv + unified requirements.txt
`agent-mesh/`	Core orchestrator, all four agents, React frontend
`src/agents/`	price_assist_agent · data_agent · rag_agent · compliance_agent · agent_factory · node_registry
`src/mesh/`	workflow.py (6-stage DAG + MeshState) · orchestrator.py (handle_request)
`src/auth/`	identity_provider.py (7 mock users) · role_permissions.py (ROLE_PERMISSIONS registry)
`src/cache/`	semantic_cache · cache_judge · entity_extractor · reranker · negative_filter · ingest_pipeline · intent_decision_store
`src/guardrails/`	deterministic_filters.py (injection · PII · destructive pattern banks)
`src/hitl/`	approval_store.py (asyncio.Event + checkpoint files)
`src/memory/`	conversation_store · jsonl_backend · redis_backend (stub) · summariser · base
`src/tracing/`	execution_trace · llm_reasoning · state_trace · cli_renderer
`src/observability/`	setup · logging_config · metrics · baggage · tracer
`src/tools/`	collaboration_tools (A2A peers + write-interceptor HITL tools)
`src/a2a/`	clients (ask_remote) · hosting (build_starlette_app + TraceContextMiddleware)
`src/integrations/`	mcp_clients (DataLayer + RAG MCP tool factories)
`frontend/src/pages/`	ChatPage · ApprovalPage · ApprovalsDashboard · AuditDashboard · FeedbackDashboard · ConversationsDashboard · LogsDashboard · MeshStatusPage
`frontend/src/components/`	MessageBubble · ExecutionPanel · LLMReasoningPanel · PipelineTrail · ApprovalModal · StructuredFeedbackModal · SecurityBadge
`workflow_evaluations/`	run_evaluation.py · evaluators/ (15 modules) · workflow/ · financial_benchmarks/ · red_team/
`data/`	audit_trail.jsonl · feedback.jsonl · conversations/ · cache_rejections.jsonl · logs/ · checkpoints/
`datalayer-as-service/`	FastMCP server + MySQL data pipeline
`mcp_server/`	server.py (18 tools) · db.py (SQLAlchemy pool) · tools.py (view queries)
`data/raw/`	14 original CSV source files
`data/curated/`	Cleaned and enriched versions of raw tables
`data/semantic/`	13 pre-joined analytical view CSVs
`sql/`	01_create_schemas · 02_create_curated_tables · 03_create_semantic_views
`rag-as-a-service/`	FastAPI + Qdrant RAG service + MCP layer
`src/gernas_rag/`	ingestion · extraction · chunking · embeddings · vectordb · retrieval · generation · evaluation · llm · cache · api/routers
`mcp_integration/`	server.py (MCP server wrapping RAG REST API on port 9000)

Path / File	Description
`docs/`	5 FAB banking PDFs (knowledge base source documents)
`scripts/`	ingest_docs · setup_vectordb · run_evaluation · verify_chunks
`my_KB/`	Project knowledge base — architecture, features, evaluation, planning docs
`fab-venv/`	Shared Python 3.13 virtual environment (short path for Windows MAX_PATH)
`requirements.txt`	Unified pinned requirements covering all three services